```
{
 "cells": [
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "# VRU-Six Separation Test\n",
    "\n",
    "Three architectures on a delayed-copy memory task — the kind of task that punishes long-sequence failure.\n",
    "\n",
    "**Models:** LSTM baseline · VRU-minimal (φ scalar only) · VRU-six (all six verticals)\n",
    "\n",
    "**Task:** signal appears in first 100 timesteps, model must reproduce it after N steps of distractor noise. Tests held memory, not local prediction.\n",
    "\n",
    "**What we're looking for:** does VRU-six hold accuracy at long delays where the others collapse.\n",
    "\n",
    "Runtime: ~15-25 min on T4. Run cells top to bottom."
   ]
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "metadata": {},
   "outputs": [],
   "source": [
    "import math\n",
    "import torch\n",
    "import torch.nn as nn\n",
    "import numpy as np\n",
    "import matplotlib.pyplot as plt\n",
    "\n",
    "device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')\n",
    "print(f'Device: {device}')\n",
    "print(f'Torch: {torch.__version__}')"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
```

```
    "## Task: delayed copy with distractors\n",
    "\n",
    "Signal in first 10 timesteps. Distractor noise for `delay` steps. Model must output the original signal at the end. Long delays = long memory required."
   ]
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "metadata": {},
   "outputs": [],
   "source": [
    "SIGNAL_LEN = 10\n",
    "SIGNAL_DIM = 4\n",
    "\n",
    "def make_batch(batch_size, delay, device):\n",
    "    \"\"\"Returns (x, y, output_mask). x has signal + distractor + recall_cue.\n",
    "    y is the signal repeated at recall positions. Mask says where to score.\"\"\"\n",
    "    total_len = SIGNAL_LEN + delay + SIGNAL_LEN\n",
    "    x = torch.zeros(total_len, batch_size, SIGNAL_DIM + 1, device=device)\n",
    "    y = torch.zeros(total_len, batch_size, SIGNAL_DIM, device=device)\n",
    "    mask = torch.zeros(total_len, batch_size, device=device)\n",
    "\n",
    "    # signal phase: random signal in first SIGNAL_DIM channels\n",
    "    signal = torch.randn(SIGNAL_LEN, batch_size, SIGNAL_DIM, device=device)\n",
    "    x[:SIGNAL_LEN, :, :SIGNAL_DIM] = signal\n",
    "\n",
    "    # distractor phase: noise\n",
    "    x[SIGNAL_LEN:SIGNAL_LEN+delay, :, :SIGNAL_DIM] = torch.randn(delay, batch_size, SIGNAL_DIM, device=device) * 0.3\n",
    "\n",
    "    # recall cue: last channel = 1 during recall phase\n",
    "    x[SIGNAL_LEN+delay:, :, SIGNAL_DIM] = 1.0\n",
    "\n",
    "    # target: reproduce signal at recall positions\n",
    "    y[SIGNAL_LEN+delay:, :, :] = signal\n",
    "    mask[SIGNAL_LEN+delay:, :] = 1.0\n",
    "\n",
    "    return x, y, mask\n",
    "\n",
    "# sanity check\n",
    "x, y, m = make_batch(2, 100, device)\n",
    "print(f'Shapes: x={x.shape}, y={y.shape}, mask={m.shape}')\n",
```

```
  "print(f'Mask sum (should be 2*10=20): {m.sum().item()}')"
 ]
},
{
 "cell_type": "markdown",
 "metadata": {},
 "source": [
  "## Models"
 ]
},
{
 "cell_type": "code",
 "execution_count": null,
 "metadata": {},
 "outputs": [],
 "source": [
  "PHI = 4.0 / math.pi\n",
  "PI_RATIO = math.pi / 4.0\n",
  "HIDDEN = 64\n",
  "INPUT_DIM = SIGNAL_DIM + 1\n",
  "OUTPUT_DIM = SIGNAL_DIM\n",
  "\n",
  "class LSTMBaseline(nn.Module):\n",
  "    def __init__(self):\n",
  "        super().__init__()\n",
  "        self.lstm = nn.LSTM(INPUT_DIM, HIDDEN)\n",
  "        self.out = nn.Linear(HIDDEN, OUTPUT_DIM)\n",
  "    def forward(self, x):\n",
  "        h, _ = self.lstm(x)\n",
  "        return self.out(h)\n",
  "\n",
  "class VRUMinimal(nn.Module):\n",
  "    \"\"\"Just the phi scalar. No verticals.\"\"\"\n",
  "    def __init__(self):\n",
  "        super().__init__()\n",
  "        self.Wx = nn.Linear(INPUT_DIM, HIDDEN)\n",
  "        self.Wh = nn.Linear(HIDDEN, HIDDEN)\n",
  "        self.out = nn.Linear(HIDDEN, OUTPUT_DIM)\n",
  "    def forward(self, x):\n",
  "        T, B, _ = x.shape\n",
  "        h = torch.zeros(B, HIDDEN, device=x.device)\n",
  "        outs = []\n",
  "        for t in range(T):\n",
```

```
   "            h = torch.tanh(self.Wx(x[t]) + PHI * self.Wh(h))\n",
   "            outs.append(self.out(h))\n",
   "        return torch.stack(outs)"
  ]
 },
 {
  "cell_type": "code",
  "execution_count": null,
  "metadata": {},
  "outputs": [],
  "source": [
   "class VRUSix(nn.Module):\n",
   "    \"\"\"\n",
   "    Six verticals:\n",
   "      1. Memory lattice — external state, read by geometric proximity\n",
   "      2. Curvature modulation — phi shifts with local information density\n",
   "      3. Dual-path coupling — h at phi, g at pi_ratio, geometrically mixed\n",
   "      4. Invariant tracking — drift correction toward anchor magnitude\n",
   "      5. Boundary conditions — learnable entry anchors\n",
   "      6. Enclosing geometry — outgoing ping reflects off boundary, returns as input\n",
   "    \"\"\"\n",
   "    def __init__(self):\n",
   "        super().__init__()\n",
   "        # dual paths\n",
   "        self.Wx_h = nn.Linear(INPUT_DIM, HIDDEN)\n",
   "        self.Wh_h = nn.Linear(HIDDEN, HIDDEN)\n",
   "        self.Wx_g = nn.Linear(INPUT_DIM, HIDDEN)\n",
   "        self.Wg_g = nn.Linear(HIDDEN, HIDDEN)\n",
   "        # cross-path coupling (geometric, no gating)\n",
   "        self.couple_hg = nn.Linear(HIDDEN, HIDDEN, bias=False)\n",
   "        self.couple_gh = nn.Linear(HIDDEN, HIDDEN, bias=False)\n",
   "        # memory lattice — fixed slot count, positions in hidden space\n",
   "        self.lattice_size = 16\n",
   "        self.M = nn.Parameter(torch.randn(self.lattice_size, HIDDEN) * 0.1)\n",
   "        self.lattice_write = nn.Linear(HIDDEN, HIDDEN)\n",
   "        # boundary anchors (vertical 5)\n",
   "        self.anchor_h = nn.Parameter(torch.randn(HIDDEN) * 0.1 + 1.27 /
math.sqrt(HIDDEN))\n",
   "        self.anchor_g = nn.Parameter(torch.randn(HIDDEN) * 0.1 + 0.79 /
math.sqrt(HIDDEN))\n",
   "        # echo / enclosing geometry (vertical 6)\n",
   "        self.echo_out = nn.Linear(HIDDEN, HIDDEN)\n",
   "        self.echo_return = nn.Linear(HIDDEN, HIDDEN)\n",
```

```
    "        # output combines h, g, and lattice signature\n",
    "        self.out = nn.Linear(HIDDEN * 2 + self.lattice_size, OUTPUT_DIM)\n",
    "\n",
    "    def forward(self, x):\n",
    "        T, B, _ = x.shape\n",
    "        # vertical 5: boundary entry\n",
    "        h = self.anchor_h.unsqueeze(0).expand(B, -1).contiguous()\n",
    "        g = self.anchor_g.unsqueeze(0).expand(B, -1).contiguous()\n",
    "        # invariant anchor — what magnitude should stay near\n",
    "        invariant_target = (self.anchor_h.pow(2).sum() +
self.anchor_g.pow(2).sum()).detach()\n",
    "\n",
    "        prev_mag = torch.zeros(B, device=x.device)\n",
    "        outs = []\n",
    "        for t in range(T):\n",
    "            xt = x[t]  # [B, INPUT_DIM]\n",
    "\n",
    "            # vertical 2: curvature modulation from local information density\n",
    "            mag = xt.pow(2).sum(dim=-1)  # [B]\n",
    "            delta = (mag - prev_mag).abs()  # [B]\n",
    "            prev_mag = mag\n",
    "            phi_dyn = PHI / (1.0 + 0.1 * delta)  # [B]\n",
    "            pi_dyn = PI_RATIO * (1.0 + 0.1 * delta)  # [B]\n",
    "\n",
    "            # vertical 1: lattice read by cosine proximity to h\n",
    "            h_norm = h / (h.norm(dim=-1, keepdim=True) + 1e-8)  # [B, HIDDEN]\n",
    "            M_norm = self.M / (self.M.norm(dim=-1, keepdim=True) + 1e-8)  # [L, HIDDEN]\n",
    "            proximity = h_norm @ M_norm.t()  # [B, L]\n",
    "            # also write current state into nearest lattice slot (soft)\n",
    "            write_strength = torch.softmax(proximity * 4.0, dim=-1)  # [B, L]\n",
    "            # additive write — we don't mutate self.M (would break batching),\n",
    "            # instead the proximity vector itself carries the signature\n",
    "            lattice_sig = proximity  # [B, L]\n",
    "\n",
    "            # vertical 6: echo / enclosing geometry\n",
    "            # h pings outward, hits tanh boundary (the wall of the enclosure),\n",
    "            # reflected signal returns as input to the next step\n",
    "            outgoing = self.echo_out(h)\n",
    "            wall = torch.tanh(outgoing)  # the enclosure boundary\n",
    "            echo = self.echo_return(wall)\n",
    "\n",
    "            # vertical 3: dual-path update with geometric mixing\n",
    "            h_pre = self.Wx_h(xt) + phi_dyn.unsqueeze(-1) * self.Wh_h(h) + self.couple_gh(g) +
```

```
0.15 * echo\n",
    "            g_pre = self.Wx_g(xt) + pi_dyn.unsqueeze(-1) * self.Wg_g(g) + self.couple_hg(h)\n",
    "            h_new = torch.tanh(h_pre)\n",
    "            g_new = torch.tanh(g_pre)\n",
    "\n",
    "            # vertical 4: invariant correction toward anchor magnitude\n",
    "            current_mag = h_new.pow(2).sum(dim=-1) + g_new.pow(2).sum(dim=-1)  # [B]\n",
    "            ratio = torch.sqrt(invariant_target / (current_mag + 1e-6))  # [B]\n",
    "            # soft correction — pull, don't snap\n",
    "            correction = 0.85 + 0.15 * ratio  # [B]\n",
    "            h = h_new * correction.unsqueeze(-1)\n",
    "            g = g_new * correction.unsqueeze(-1)\n",
    "\n",
    "            feature = torch.cat([h, g, lattice_sig], dim=-1)\n",
    "            outs.append(self.out(feature))\n",
    "\n",
    "        return torch.stack(outs)\n",
    "\n",
    "# parameter count comparison\n",
    "for cls in [LSTMBaseline, VRUMinimal, VRUSix]:\n",
    "    m = cls()\n",
    "    n = sum(p.numel() for p in m.parameters())\n",
    "    print(f'{cls.__name__}: {n:,} params')"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "## Training\n",
    "\n",
    "Train each model on the copy task across a range of delays. Track recall MSE at each delay length."
   ]
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "metadata": {},
   "outputs": [],
   "source": [
    "def masked_mse(pred, y, mask):\n",
    "    diff = (pred - y).pow(2).sum(dim=-1)  # [T, B]\n",
```

```
    "    return (diff * mask).sum() / (mask.sum() + 1e-8)\n",
    "\n",
    "def train_model(model_cls, delays_train, n_steps=2000, batch_size=16, lr=3e-3,
seed=0):\n",
    "    torch.manual_seed(seed)\n",
    "    np.random.seed(seed)\n",
    "    model = model_cls().to(device)\n",
    "    opt = torch.optim.Adam(model.parameters(), lr=lr)\n",
    "    losses = []\n",
    "    for step in range(n_steps):\n",
    "        # curriculum: sample delay from training range\n",
    "        delay = int(np.random.choice(delays_train))\n",
    "        x, y, mask = make_batch(batch_size, delay, device)\n",
    "        pred = model(x)\n",
    "        loss = masked_mse(pred, y, mask)\n",
    "        opt.zero_grad()\n",
    "        loss.backward()\n",
    "        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)\n",
    "        opt.step()\n",
    "        losses.append(loss.item())\n",
    "        if (step + 1) % 200 == 0:\n",
    "            recent = np.mean(losses[-100:])\n",
    "            print(f'  step {step+1:4d}  loss {recent:.5f}')\n",
    "    return model, losses\n",
    "\n",
    "def eval_model(model, delays_test, batch_size=32, n_batches=4):\n",
    "    model.eval()\n",
    "    results = {}\n",
    "    with torch.no_grad():\n",
    "        for delay in delays_test:\n",
    "            losses = []\n",
    "            for _ in range(n_batches):\n",
    "                x, y, mask = make_batch(batch_size, delay, device)\n",
    "                pred = model(x)\n",
    "                losses.append(masked_mse(pred, y, mask).item())\n",
    "            results[delay] = np.mean(losses)\n",
    "    return results"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "metadata": {},
```

```
 "outputs": [],
 "source": [
  "# train on short-medium delays, test on full range including extrapolation\n",
  "DELAYS_TRAIN = [50, 100, 200, 400]\n",
  "DELAYS_TEST = [50, 100, 200, 400, 800, 1500, 3000]\n",
  "\n",
  "all_results = {}\n",
  "\n",
  "for cls in [LSTMBaseline, VRUMinimal, VRUSix]:\n",
  "    name = cls.__name__\n",
  "    print(f'\\n=== Training {name} ===')\n",
  "    model, _ = train_model(cls, DELAYS_TRAIN, n_steps=2000, seed=0)\n",
  "    print(f'  evaluating...')\n",
  "    res = eval_model(model, DELAYS_TEST)\n",
  "    all_results[name] = res\n",
  "    for d, v in res.items():\n",
  "        print(f'    delay {d:5d}: {v:.5f}')"
 ]
},
{
 "cell_type": "code",
 "execution_count": null,
 "metadata": {},
 "outputs": [],
 "source": [
  "# results table\n",
  "print(f\"{'Delay':>8} | {'LSTM':>10} | {'VRU-min':>10} | {'VRU-six':>10}\")\n",
  "print('-' * 50)\n",
  "for d in DELAYS_TEST:\n",
  "    lstm_v = all_results['LSTMBaseline'][d]\n",
  "    vmin_v = all_results['VRUMinimal'][d]\n",
  "    vsix_v = all_results['VRUSix'][d]\n",
  "    marker = ''\n",
  "    if vsix_v < lstm_v and vsix_v < vmin_v:\n",
  "        marker = '  <-- VRU-six best'\n",
  "    print(f'{d:>8} | {lstm_v:>10.5f} | {vmin_v:>10.5f} | {vsix_v:>10.5f}{marker}')\n",
  "\n",
  "# plot\n",
  "fig, ax = plt.subplots(figsize=(10, 6))\n",
  "for name in ['LSTMBaseline', 'VRUMinimal', 'VRUSix']:\n",
  "    ys = [all_results[name][d] for d in DELAYS_TEST]\n",
  "    ax.plot(DELAYS_TEST, ys, marker='o', label=name)\n",
  "ax.set_xlabel('Delay (timesteps)')\n",
```

```
    "ax.set_ylabel('Recall MSE')\n",
    "ax.set_yscale('log')\n",
    "ax.set_xscale('log')\n",
    "ax.axvspan(50, 400, alpha=0.1, color='green', label='training range')\n",
    "ax.legend()\n",
    "ax.set_title('Recall accuracy vs delay length')\n",
    "ax.grid(True, alpha=0.3)\n",
    "plt.tight_layout()\n",
    "plt.show()"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "## Multi-seed run (run if first results are interesting)\n",
    "\n",
    "Single seed = single trajectory. If the first run shows separation, run this to check it's not just lucky initialization."
   ]
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "metadata": {},
   "outputs": [],
   "source": [
    "SEEDS = [0, 1, 2]\n",
    "multi_results = {name: {d: [] for d in DELAYS_TEST} for name in ['LSTMBaseline', 'VRUMinimal', 'VRUSix']}\n",
    "\n",
    "for seed in SEEDS:\n",
    "    print(f'\\n--- seed {seed} ---')\n",
    "    for cls in [LSTMBaseline, VRUMinimal, VRUSix]:\n",
    "        name = cls.__name__\n",
    "        print(f'  {name}...')\n",
    "        model, _ = train_model(cls, DELAYS_TRAIN, n_steps=2000, seed=seed)\n",
    "        res = eval_model(model, DELAYS_TEST)\n",
    "        for d, v in res.items():\n",
    "            multi_results[name][d].append(v)\n",
    "\n",
    "print(f\"\\n{'Delay':>8} | {'LSTM (mean±std)':>20} | {'VRU-min (mean±std)':>22} | {'VRU-six (mean±std)':>22}\")\n",
```

```
   "print('-' * 80)\n",
   "for d in DELAYS_TEST:\n",
   "    row = f'{d:>8} | '\n",
   "    for name in ['LSTMBaseline', 'VRUMinimal', 'VRUSix']:\n",
   "        vals = multi_results[name][d]\n",
   "        mean = np.mean(vals)\n",
   "        std = np.std(vals)\n",
   "        row += f'{mean:.5f} ± {std:.5f} | '\n",
   "    print(row)"
  ]
 }
],
"metadata": {
 "kernelspec": {
  "display_name": "Python 3",
  "language": "python",
  "name": "python3"
 },
 "language_info": {
  "name": "python",
  "version": "3.10"
 }
},
"nbformat": 4,
"nbformat_minor": 4
}
```